

实验七 查找实验

基础练习：练习 1、2

进阶练习：练习 1、2

基础练习

【注意】本实验中，所有表元素均具有唯一关键字值。

【练习一】填空：在一个无序表 A 中采用顺序查找算法查找值为 x 的元素。

```
# include <stdio.h>
int search (int A[], int x, int n)
{
    int i=0;
    while (i<n && A[i]!=x)
        i++;
    if (i>=n)
        return _____;
    else
        return _____ ;
}
void main()
{
    int a[]={2, 5, 56, 10, 12, 15, 8, 19, 25, 32}, n, d, i;
    printf ("A 数值\n 下标");
    for (i=0; i<10; i++)
        printf ("%3d",i);
    printf ("\n 值 ");
    for (i=0; i<10; i++)
        printf ("%3d",a[i]);
    printf ("\n 输入值: ");
    scanf ("%d",&d);
    n=search(a, d, 10);
    if (n>=0)
        printf ("A[%d]=\n",d);
    else
        printf ("%d 未找到\n",d);
}
```

【练习二】一个长度为 n ($n \leq 1024$) 的有序表，其元素分别包含一个整型键值。编写程序：查找键值为 x 的元素。

【练习三】设有关键字集合：{22, 12, 13, 8, 9, 20, 33, 42, 44, 38, 24, 48, 60, 58, 74, 49, 86, 53}，为其建立索引表，实现分块查找算法。

【提示】分块查找又称为索引顺序查找。分块查找过程分两步进行：首先用某种查找方法查找索引表（若索引表有序，则采用二分查找检索索引表），以确定待查找数据属于哪一块。然后在块内顺序查找要找的数据。本例可分 3 块建立查找表及其索引表，分块结果是块内无序，块间有序。索引表结点结构中包括：块的起始地址与块内的最大关键字。

3 进阶练习

【练习一】一个长度为 n ($n \leq 1024$) 的有序表，其元素分别包含一个整型键值。编写程序：向有序表中插入一个元素 x ，并保持表的有序性。

【提示】利用二分查找方法寻找 x 的插入位置。

【练习二】编程，有 n ($n \leq 100$) 个整数，构造其对应的二叉排序树。

【提示】二叉排序树的生成，可从空的二叉树开始，每输入一个结点数据，就建立一个新结点插入到当前已生成的二叉排序树中，所以它的主要操作是二叉排序树的插入运算。在二叉排序树中插入新结点，只要保证插入后仍符合二叉排序树的定义即可。

插入新结点的过程：若二叉排序树为空，则待插入结点 s （结点由指针 s 所指向）作为根结点插入到空树中；当二叉排序树非空，将待插结点的关键字 $s \rightarrow \text{key}$ 与树根的关键字 $t \rightarrow \text{key}$ 比较，若 $s \rightarrow \text{key} = t \rightarrow \text{key}$ ，则说明树中已有此结点，无需插入；若 $s \rightarrow \text{key} < t \rightarrow \text{key}$ ，则将待插结点 s 插入到根的左子树中，否则将 s 插入到根的右子树中。而子树中的插入过程又和在树中的插入过程相同，如此进行下去，直到把结点 s 作为一个新的树叶插入到二叉排序树中，或者直到发现树中已有结点 s 为止。